

Documentation for `nstat.chpl`

Nelson Luís Dias (nelsonluisdias@gmail.com)

September 2, 2026

Notation

In this document,

$$E\{X\} = \int_{\mathbb{R}} x f_X(x) \, dx, \quad (1)$$

$$\text{Var}\{X\} = \int_{\mathbb{R}} (x - E\{x\})^2 f_X(x) \, dx \quad (2)$$

(where $f_X(x)$ is the probability density function of X) are population parameters, whereas

$$E\{x\} = \bar{x} = \frac{1}{n} \sum_{i=1}^n x_i, \quad (3)$$

$$\text{Var}\{x\} = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2 \quad (4)$$

are sample statistics.

1 `stat1`: mean of an array

1.1 Use

```
proc stat1(  
    const ref x: [] real          // the data  
): real                          // mean  
//   where x.rank == 1
```

1.2 Theory

Calculates

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i, \quad (5)$$

where n is the size of the array x .

2 stat2: mean and biased variance of an array

2.1 Use

```
proc stat2(  
  const ref x: [] real      // the data  
): (real, real)             // mean, variance  
  where x.rank == 1 {  
  }
```

2.2 Theory

Calculates

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i, \quad (6)$$

$$\text{Var}\{x\} = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2. \quad (7)$$

where n is the size of the array x .

3 nanstat1: mean, ignoring nans

3.1 Use

```
proc nanstat1(  
  ref x: [] real            // the data  
): (int, real)              // number of valid data, mean  
  where x.rank == 1 {  
  }
```

3.2 Theory

Calculates

$$\delta_i = \begin{cases} 1 & x_i \neq \text{NaN}, \\ 0 & x_i = \text{NaN}, \end{cases} \quad (8)$$

$$m = \sum_{i=1}^n \delta_i, \quad (9)$$

$$\bar{x} = \frac{1}{m} \sum_{i=1}^n \delta_i x_i. \quad (10)$$

4 nanstat2: mean and biased variance, ignoring nans

4.1 Use

```

proc nanstat2(
  ref x: [] real          // the data
): (int,real,real) {      // number of valid data, mean, variance
  const n = x.size;

```

4.2 Theory

Calculates

$$\delta_i = \begin{cases} 1 & x_i \neq \text{NaN}, \\ 0 & x_i = \text{NaN}. \end{cases} \quad (11)$$

$$m = \sum_{i=1}^n \delta_i, \quad (12)$$

$$\bar{x} = \frac{1}{m} \sum_{i=1}^n \delta_i x_i, \quad (13)$$

$$\text{Var}\{x\} = \frac{1}{m} \sum_{i=1}^n (\delta_i x_i - \bar{x})^2. \quad (14)$$

5 wstat1: weighted mean

5.1 Use

```

proc wstat1(
  const ref x: [] real,      // the data
  const ref w: [] ?tw        // the weights
): real                      // the weighted mean

```

5.2 Theory

Calculates

$$W = \sum_{i=1}^n w_i, \quad (15)$$

$$\bar{x} = \frac{1}{W} \sum_{i=1}^n w_i x_i, \quad (16)$$

where w_i are the weights of the data points x_i , and n is the size of the arrays $\mathbf{w}$ and $\mathbf{x}$.

6 wstat2: weighted mean and variance

6.1 Use

```

proc wstat2(
  const ref x: [] real,      // the data
  const ref w: [] ?tw       // the weights
): (real, real)             // mean, variance

```

6.2 Theory

Calculates

$$W = \sum_{i=1}^n w_i, \quad (17)$$

$$\bar{x} = \frac{1}{W} \sum_{i=1}^n w_i x_i, \quad (18)$$

$$\text{Var}\{x\} = \frac{1}{W} \sum_{i=1}^n w_i (x_i - \bar{x})^2 \quad (19)$$

where w_i are the weights of the data points x_i , and n is the size of the arrays $\mathbf{w}$ and $\mathbf{x}$.

7 nanwstat1: weighted mean, ignoring NaNs

7.1 Use

```

proc nanwstat1(
  const ref x: [] real,      // the data
  const ref w: [] ?tw       // the weights
): (int, real)              // number of valid data points, weighted mean

```

7.2 Theory

Calculates

$$\delta_i = \begin{cases} 1 & x_i \neq \text{NaN}, \\ 0 & x_i = \text{NaN}. \end{cases} \quad (20)$$

$$W = \sum_{i=1}^n \delta_i w_i, \quad (21)$$

$$\bar{x} = \frac{1}{W} \sum_{i=1}^n w_i \delta_i x_i, \quad (22)$$

$$(23)$$

where w_i are the weights of the data points x_i , and n is the size of the arrays $\mathbf{w}$ and $\mathbf{x}$.

8 covar: the covariance

8.1 Use

```
proc covar(  
  const in xm: real,          // the mean of x  
  const in ym: real,          // the mean of y  
  const ref x: [] real,       // the x data  
  const ref y: [] real        // the y data  
): real                        // the covariance  
  where (x.rank == 1 && y.rank == 1) {
```

8.2 Theory

Calculates

$$\text{Cov}\{x, y\} = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}), \quad (24)$$

where $\bar{x}$ is the mean of x , $\bar{y}$ is the mean of y , and n is their size. Note that you must calculate $\bar{x}$ and $\bar{y}$ *before* calling covar.

9 trivar: the trivariance

9.1 Use

```
proc trivar(  
  const in xm: real,          // the mean of x  
  const in ym: real,          // the mean of y  
  const in zm: real,          // the mean of z  
  const ref x: [] real,       // the x data  
  const ref y: [] real,       // the y data  
  const ref z: [] real        // the z data  
): real                        // the trivariance  
  where (x.rank == 1) && (y.rank == 1) && (z.rank == 1) {
```

9.2 Theory

Calculates

$$\text{Trv}\{x, y, z\} = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})(z_i - \bar{z}), \quad (25)$$

where $\bar{x}$ is the mean of x , $\bar{y}$ is the mean of y , $\bar{z}$ is the mean of z , and n is their size. Note that you must calculate $\bar{x}$, $\bar{y}$ and $\bar{z}$ *before* calling covar.

10 median: the median

10.1 Use

```
proc median(  
  const ref ax: [] ?at          // the data  
): at                          // the median  
  where (isNumericType(at) && (!isComplex(at))) && ax.rank == 1) {
```

10.2 Theory

Calculates the median of the data in array $x = (x_0, x_1, \dots, x_{n-1})$. The array is sorted by an index array I such that x_{I_0} is the minimum value and $x_{I_{n-1}}$ is the maximum value. Returns

$$x_{\text{median}} = \begin{cases} \frac{1}{2} (x_{I_{n/2-1}} + x_{I_{n/2}}) & n \text{ is even,} \\ x_{I_{n/2}} & n \text{ is odd.} \end{cases} \quad (26)$$

The domain of ax *does not have to start at 0*, because `median` takes care to reindex it internally.

11 nanmedian: the median, ignoring NaNs

11.1 Use

```
proc nanmedian(  
  const ref ax: [] ?at          // the data (may include NaNs)  
): at                          // the median  
  where (isNumericType(at) && (!isComplex(at))) && ax.rank == 1) {
```

11.2 Theory

Calculates the median of the data in an array. First, all NaNs are purged from the array. Then, the (resulting) array $x = (x_0, x_1, \dots, x_{n-1})$ is sorted by an index array I such that x_{I_0} is the minimum value and $x_{I_{n-1}}$ is the maximum value. Returns

$$x_{\text{median}} = \begin{cases} \frac{1}{2} (x_{I_{n/2-1}} + x_{I_{n/2}}) & n \text{ is even,} \\ x_{I_{n/2}} & n \text{ is odd.} \end{cases} \quad (27)$$

The domain of ax *does not have to start at 0*, because `nanmedian` takes care to reindex it internally.

12 quartiles: returns the quartiles of an array

12.1 Use

```
proc quartiles(  
  const in ax: [] real          // we need to make a copy!  
): (real, real, real)          // the 3 quartiles  
  where ax.rank == 1 {
```

12.2 Theory

Makes a copy of the array; sorts it; the cumulative distribution function over the sorted array $(x_0, x_1, \dots, x_{n-1})$ is

$$F(x_i) = \frac{i}{n-1}. \quad (28)$$

Then interpolates

$$q_j = F^{-1}(j \times 0.25), \quad j = 1, 2, 3, \quad (29)$$

and returns (q_1, q_2, q_3) .

13 gmean: geometric mean

13.1 Use

```
proc gmean(  
  ref x: [] real                                // the data  
) : real                                       // the geometric mean  
where x.rank == 1 {  
  }
```

13.2 Theory

Given a sequence $(x_1, x_2, \dots, x_n)$ of *strictly positive real numbers*, the geometric mean is

$$\tilde{x} = \left[\prod_{i=1}^n x_i \right]^{1/n} \quad (30)$$

For large n and small individual values of x_i , however, (30) can lead to underflow. It is much more robust to write

$$\begin{aligned} \tilde{x} &= \exp \left\{ \ln \left[\left(\prod_{i=1}^n x_i \right)^{1/n} \right] \right\} \\ &= \exp \left[\frac{1}{n} \sum_{i=1}^n \ln(x_i) \right]. \end{aligned} \quad (31)$$

14 gmom: geometric-modulus mean

14.1 Use

```
proc gmom(  
  ref x: [] real                                // the array being averaged  
) : real                                       // the geometric-modulus mean  
where x.rank == 1 {  
  }
```

14.2 Theory

gmom is my own humble attempt to generalize the idea of a geometric mean to sequences containing negative and null values, as well as positive.

First, consider a sequence $(x_1, x_2, \dots, x_n)$ containing only negative values. It is reasonable to define its “geometric” mean as

$$\tilde{x} = -\exp \left[\frac{1}{n} \sum_{i=1}^n \ln(|x_i|) \right]. \quad (32)$$

For a sequence containing negative, null, and positive values, $\ln(\cdot)$ can no longer be used, and the singularity at the origin needs to be avoided. A function that has those attributes and is asymptotically equal to $\ln(\cdot)$ for large positive values of its argument is the *log-modulus* function introduced by [John and Draper \(1980\)](#):

$$\text{logmod}(x) \equiv \text{sign}(x) \ln(|x| + 1). \quad (33)$$

Note that, asymptotically,

$$\text{logmod}(x) \sim \begin{cases} x, & x \sim 0, \\ \text{sign}(x) \ln(|x|), & x \gg 1, \end{cases} \quad (34)$$

and that $\text{logmod}(x)$ is odd.

In order to mimic (31), we need the inverse of logmod :

$$\begin{aligned} y &= \text{sign}(x) \ln(|x| + 1), \\ \text{sign}(y)y &= \ln(|x| + 1), \end{aligned}$$

(because logmod is odd)

$$\begin{aligned} \exp(\text{sign}(y)y) &= |x| + 1, \\ \exp(|y|) &= |x| + 1, \\ |x| &= \exp(|y|) - 1, \\ x &= \text{sign}(y) [\exp(|y|) - 1], \end{aligned}$$

whence

$$\text{expmod}(x) \equiv \text{sign}(x) [\exp(|x|) - 1] \quad (35)$$

is the *exponential-modulus* function.

Note: the log-modulus and exponential-modulus functions are defined in `nmth.chpl` and imported into `nstat.chpl`.

We are now ready to define the *geometric-modulus* mean:

$$\tilde{x} \equiv \text{expmod} \left[\frac{1}{n} \sum_{i=1}^n \text{logmod}(x_i) \right]. \quad (36)$$

Evidently, for a sequence of positive real values where $x_i \gg 1 \forall i = 1, \dots, n$, (36) will produce results which are very close to (31). However, if on the other hand $x_i \ll 1 \forall i = 1, \dots, n$, (36) will be very much like the arithmetic mean, which is undesirable. The solution to always obtain a “geometric mean”-like behavior for any sequence (now including possibly null and negative values as well) is to

re-scale before averaging, by means of the following sequence of operations:

$$\delta = \frac{\min(|x_i|)}{16}, \quad x_i \neq 0, \quad (37)$$

$$\xi_i = \frac{x_i}{\delta}, \quad (38)$$

$$\tilde{\xi} = \text{expmod} \left[\frac{1}{n} \sum_{i=1}^n \text{logmod}(\xi_i) \right], \quad (39)$$

$$\tilde{x} = \delta \tilde{\xi}. \quad (40)$$

The factor 16 in (37) is arbitrary and is meant to force all non-null ξ to be “sufficiently” far from ± 1 .

15 whiskers: returns the minimum, quartiles, and maximum of an array

15.1 Use

```
proc whiskers(
  const in x: [?Dx] real           // we need to make a copy!
): [0..4] real                     // an array with the whiskers
  where x.rank == 1 {
```

15.2 Theory

Makes a copy of the array; sorts it; the cumulative distribution function over the sorted array $(x_0, x_1, \dots, x_{n-1})$ is

$$F(x_i) = \frac{i}{n-1}. \quad (41)$$

Then interpolates

$$q_j = F^{-1}(j \times 0.25), \quad j = 1, 2, 3, \quad (42)$$

and returns the array $\mathbf{w} = (x_0, q_1, q_2, q_3, x_{n-1})$.

16 xbins: bins data into a ragged array

16.1 Use

```
proc xbins(
  const in xmin: real,           // lower x limit
  const in dx: real,             // size of x bin
  const in xmax: real,           // upper x limit
  const ref x: [] real,          // xdata
  const ref y: [] real           // ydata
): [] vec                       // y data in each bin
  where (x.rank == 1) && (y.rank == 1) {
```

16.2 Theory

Given the arrays $\mathbf{x}$, $\mathbf{y}$ of data points, and the bin limits $x_{bmin}, \Delta x_b, x_{bmax}$, such that

$$xb_j = xb_{min} + (j + 0.5)\Delta x_b, \quad (43)$$

returns $\mathbf{y}_b$ where y_{bj} is a (ragged) array that contains all data points y_i such that

$$xb_j - \Delta x_b \leq x_i < xb_j + \Delta x_b. \quad (44)$$

17 outlimits: defines limits to detect outliers based on the interquartile range

17.1 Use

```
proc outlimits(  
  ref x: [] real,           // the data  
  in delta: real = 1.5     // interquartile range multiplier  
) : (real, real)          // (minx, maxx)  
  where x.rank == 1 {  
  }
```

17.2 Theory

Inspired by [Merchant and Harris \(1999\)](#), [Kilpatrick et al. \(2015\)](#), etc.: given an array $\mathbf{x}$ of data and a coefficient Δ ($\Delta = 1.5$ by default, but it can be modified), proposes the limits

$$x_{min} = q_1 - \Delta(q_3 - q_1), \quad (45)$$

$$x_{max} = q_3 + \Delta(q_3 - q_1), \quad (46)$$

where (q_1, q_2, q_3) are the quartiles, $(q_3 - q_1)$ is the *interquartile range*, and (x_{min}, x_{max}) is the range outside of which data are to be considered outliers.

18 force_bounds: constrains x to the bounds xmin, xmax.

18.1 Use

```
inline proc force_bounds(  
  ref x: real,  
  const in xmin: real,  
  const in xmax: real) {  
  if x < xmin then {  
    x = xmin;  
  }  
  else if x > xmax then {  
    x = xmax;  
  }  
}
```

18.2 Theory

The code above speaks for itself.

19 reglin: linear regression

19.1 Use

```
proc reglin(  
  const ref x: [] real,      // x data  
  const ref y: [] real      // y data  
) : (real, real, real)      // (a,b,r) where y = ax + b  
  where (x.rank == 1 && y.rank == 1) {
```

19.2 Theory

Given a pair of data $\mathbf{x} = (x_1, \dots, x_n)$, $\mathbf{y} = (y_1, \dots, y_n)$, calculates the linear regression

$$\hat{y} = ax + b,$$

where

$$a = \frac{\text{Cov}\{x, y\}}{\text{Var}\{x\}}, \quad (47)$$

$$b = \bar{y} - a\bar{x}, \quad (48)$$

and

$$r = \frac{\text{Cov}\{x, y\}}{\sqrt{\text{Var}\{x\} \text{Var}\{y\}}} \quad (49)$$

is the correlation coefficient.

20 reglina: linear regression through the origin

20.1 Use

```
proc reglina(  
  const ref x: [] real,      // x data  
  const ref y: [] real,      // y data  
  out a: real,               // the slope  
  out cd: real,               // coeff of det: only cd makes sense here!  
  out se: real,               // std error of estimate  
  out sa: real                // error of estimate of a  
)  
  where (x.rank == 1 && y.rank == 1) {
```

20.2 Theory

Let

$$Y = ax + E, \quad (50)$$

where Y is a random variable, E is a random variable independent of x , and x is not random.

Form the objective function

$$F_O(a) = \frac{1}{n} \sum_{i=1}^n (y_i - ax_i)^2. \quad (51)$$

Differentiating with respect to a gives the least-squares estimate of a :

$$\begin{aligned} \sum_{i=1}^n 2(y_i - ax_i)(-x_i) &= 0, \\ -\sum_{i=1}^n x_i y_i + a \sum_{i=1}^n x_i^2 &= 0, \\ a &= \frac{\sum_{i=1}^n x_i y_i}{\sum_{i=1}^n x_i^2}. \end{aligned} \quad (52)$$

The “error” involved in estimating a can be found by re-interpreting a (and y_i) as random variables, and assuming that the Y_i ’s are independent:

$$\begin{aligned} A &= \frac{\sum_{i=1}^n x_i Y_i}{\sum_{i=1}^n x_i^2}, \\ \text{Var}\{A\} &= \frac{\sum_{i=1}^n x_i^2 \text{Var}\{Y\}}{\left[\sum_{i=1}^n x_i^2\right]^2}. \end{aligned} \quad (53)$$

Replacing the expectations with sample moments:

$$\begin{aligned} \text{Var}\{a\} &= \frac{\sum_{i=1}^n x_i^2}{\left[\sum_{i=1}^n x_i^2\right]^2} \text{Var}\{y\}, \\ &= \frac{\text{Var}\{y\}}{\sum_{i=1}^n x_i^2}; \end{aligned} \quad (54)$$

$$s_a = \sqrt{\frac{\text{Var}\{y\}}{\sum_{i=1}^n x_i^2}}. \quad (55)$$

The standard error of estimate shouldn’t be too difficult to find, now:

$$\begin{aligned} s_e^2 &= \frac{1}{n} \sum_{i=1}^n [y_i - ax_i]^2 \\ &= \frac{1}{n} \sum_{i=1}^n [y_i^2 - 2ax_i y_i + a^2 x_i^2] \\ &= \frac{1}{n} \left[\sum_{i=1}^n y_i^2 - 2a \sum_{i=1}^n x_i y_i + a^2 \sum_{i=1}^n x_i^2 \right]. \end{aligned} \quad (56)$$

Finally, by definition the coefficient of determination is

$$c_d = 1 - \frac{s_e^2}{\text{Var}\{y\}}. \quad (57)$$

21 reglinpar: linear regression, with error estimates

21.1 Use

```
proc reglinpar(  
    const ref x: [] real,      // the x data  
    const ref y: [] real,      // the y data  
    out a: real,               // the a of y = ax + b  
    out b: real,               // the b of y = ax + b  
    out r: real,               // correlation coefficient  
    out se: real,              // standard error of estimate  
    out sa: real,              // standard error of a  
    out sb: real               // standard error of b  
)  
where (x.rank == 1 && y.rank == 1) {
```

21.2 Theory

We follow [Benjamin and Cornell \(1970, Chapter 4\)](#)

The general probabilistic model is

$$Y = ax + b + E, \quad (58)$$

where Y is a random variable, E is a random variable independent of x , and x is not random.

The estimates of a and b and the correlation coefficient r are already given in (47), (48) and (49). The standard error of estimate is given by Eq. (4.3.27) of [Benjamin and Cornell \(1970\)](#):

$$s_e^2 = \frac{n}{n-2} (1 - r^2) \text{Var}\{y\}. \quad (59)$$

The variances of a and b (actually assumed to be random variables A and B and then estimated by the sample statistics) are

$$s_a^2 = \frac{s_e^2}{n \text{Var}\{x\}}, \quad (60)$$

$$s_b^2 = \frac{s_e^2}{n} \left(1 + \frac{\bar{x}^2}{\text{Var}\{x\}} \right). \quad (61)$$

22 ols: Ordinary least squares

22.1 Use

```
proc ols(  
    ref x: [] real,            // independent data matrix  
    ref y: [] real,            // dependent data vector  
    out S2: real,              // variance of residuals  
    out CD: real,              // adjusted coeff. of multiple determination  
    ref c: [] real,            // array of coefficients  
    ref ye: [] real,           // vector of estimated ys  
    ref ccov: [] real,         // covariance matrix of c  
    ref axcor: [] real         // correlation matrix of x
```

```

)
where (x.rank == 2 && y.rank == 1 &&
      c.rank == 1 && ccov.rank == 2 && axcor.rank == 2) {

```

22.2 Theory

Calculates the multiple linear regression coefficients $\mathbf{c}$ of

$$[\hat{\mathbf{y}}]_{n \times 1} = [\mathbf{x}]_{n \times m} [\mathbf{c}]_{m \times 1}, \quad (62)$$

where the x data are

$$[\mathbf{x}] = \begin{bmatrix} 1 & x_{11} & x_{12} & \dots & x_{1(m-1)} \\ 1 & x_{21} & x_{22} & \dots & x_{2(m-1)} \\ & & & \ddots & \\ 1 & x_{n1} & x_{n2} & \dots & x_{n(m-1)} \end{bmatrix}. \quad (63)$$

For details, see for example Chapter 7 of [Netter et al. \(1983\)](#). Start with the corresponding equation with y data,

$$[\mathbf{y}]_{n \times 1} = [\mathbf{x}]_{n \times m} [\mathbf{c}]_{m \times 1}, \quad (64)$$

which cannot be solved for $[\mathbf{c}]$, and do:

$$\begin{aligned} [\mathbf{x}]_{m \times n}^T [\mathbf{y}]_{n \times 1} &= [\mathbf{x}]_{m \times n}^T \underbrace{[\mathbf{x}]_{n \times m} [\mathbf{c}]_{m \times 1}}_{[\mathbf{A}]_{m \times m}} \\ &= [\mathbf{A}]_{m \times m} [\mathbf{c}]_{m \times 1} \Rightarrow \\ [\mathbf{c}]_{m \times 1} &= [\mathbf{A}]_{m \times m}^{-1} [\mathbf{x}]_{m \times n}^T [\mathbf{y}]_{n \times 1} \blacksquare \end{aligned} \quad (65)$$

The standard error of estimate is

$$s_e^2 = \frac{1}{n - m} [[\mathbf{y}] - [\hat{\mathbf{y}}]]^T [[\mathbf{y}] - [\hat{\mathbf{y}}]]; \quad (66)$$

the covariance matrix of $\mathbf{c}$ is

$$[\text{Cov}\{\mathbf{c}\}] = s_e^2 [\mathbf{A}]^{-1}, \quad (67)$$

and the adjusted coefficient of multiple determination ([Netter et al., 1983](#), Eq. (7.33)) is

$$C_D = 1 - \frac{n - 1}{n} \frac{s_e^2}{\text{Var}\{y\}}. \quad (68)$$

23 init_lowess: initialize data for further lowess estimates

23.1 Use

```

proc init_lowess(
  ref xx: [] real,          // the raw x data
  ref yy: [] real,          // the raw y data
  const in win: int,        // the window, once and for all, of this loess
  const in deg: int = 1     // the degree of this loess
) where (xx.rank == 1) && (yy.rank == 1) {

```

23.2 Theory

Let $\mathbf{x}$ and $\mathbf{y}$ be two data arrays containing data points (x_i, y_i) , $i = 1 \dots, n_d$. `init_lowess` simply calculates normalized $\bar{\mathbf{x}}$ and $\bar{\mathbf{y}}$:

$$x_{\min} = \min_i x_i, \quad (69)$$

$$x_{\max} = \max_i x_i, \quad (70)$$

$$\bar{x}_i = \frac{x_i - x_{\min}}{x_{\max} - x_{\min}}, \quad (71)$$

and similarly for $\mathbf{y}$. For this purpose, it calls the private procedure `normalize_array` and stores $\bar{\mathbf{x}}$ and $\bar{\mathbf{y}}$ in the private arrays `n_xx` and `n_yy`. It also stores, for further use, the size `win` of the data window for local estimates (hereafter, m), and the degree `deg` of the least-squares regressions in `lowess_estimate` (hereafter, n). All the rest is done by the next procedure, `lowess_estimate`.

24 `lowess_estimate`: estimate $\hat{y}$ at a point x using ordinary least squares

24.1 Use

```
proc lowess_estimate(
    in x: real                // point around which to do linear regression
): real {
```

24.2 Theory

Based on the “observational” data $\mathbf{x}$ and $\mathbf{y}$ passed to `init_lowess`, given an abscissa x (which may or may not be equal to one of the data points x_i to be used by `lowess`), we want to obtain a local estimate $\hat{y}$ “through” the data points.

First, x itself is normalized:

$$\bar{x} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}. \quad (72)$$

Now, for all (normalized) data points $\bar{x}_i$, calculate the distances to $\bar{x}$:

$$d_i = |\bar{x} - \bar{x}_i|, \quad i = 1, \dots, n_d. \quad (73)$$

This will be used to calculate the weights of the m nearest points to x . We need the indices of the m x_i s closest to x . This is done by the private procedure `get_min_range`. The indices of the m nearest neighbors are found by `indxquickselect`, from module `ssr`, and returned in the array of `ints` `min_range` (hereafter R), together with the maximum distance among them $d_{\max}$ (in variable `maxdist`):

$$d_{\max} = \max d_{R_i}, \quad i = 1, \dots, m. \quad (74)$$

This allows the calculation of the weights of each data point x_{R_i} , $i = 1, \dots, m$:

$$w(x) \equiv (1 - |x|^3)^3, \quad (75)$$

$$w_i = w(d_{R_i}/d_{\max}). \quad (76)$$

The w_i s will be put in the diagonal of a matrix W for weighted least squares. The statistical model is

$$\widehat{Y} = \beta_0 + \beta_1 \bar{x} + \beta_2 \bar{x}^2 + \dots + \beta_n \bar{x}^n. \quad (77)$$

We apply the relevant theory without details. Build the matrices $[W]_{m \times m}$ and $[X]_{m \times n}$:

$$W_{ij} = \delta_{ij} w_{(i)}, \quad (78)$$

$$X_{i,0} = 1, \quad (79)$$

$$X_{i,1} = \bar{x}_{R_i} \quad (80)$$

$$X_{i,j} = \bar{x}_{R_i}^j. \quad (81)$$

Note that if $n = 1$, the last line above is not calculated. In (78), δ_{ij} is Kronecker's delta. The matrix operations in `smatrix.chpl` involving $[W]$, therefore, use a 1-dimensional array to store only its diagonal: see `dot_m_diagm` and `dot_mt_diagm`. The observed normalized ordinates are stored in vector $[Y]_{m \times 1}$, with

$$Y_i = \bar{y}_{R_i}. \quad (82)$$

Now let $[A]_{n+1 \times m}$, $[B]_{n+1 \times n+1}$, $[C]_{n+1 \times m}$ be matrices, $[\beta]_{n+1 \times 1}$ be a vector, and calculate a standard weighted multiple regression:

$$[A] \leftarrow [X]^T [W],$$

$$[B] \leftarrow [A][X],$$

$$[B] \leftarrow [B]^{-1},$$

$$[A] \leftarrow [B][X]^T,$$

$$[C] \leftarrow [A][W],$$

$$[\beta] \leftarrow [C][Y].$$

where $\beta_0, \beta_1, \dots, \beta_n$ are the regression parameters. Then,

$$\widehat{Y} = [\beta]^T [X] \quad (83)$$

is the normalized estimate and

$$\widehat{y} = y_{\min} + \widehat{Y} \times (y_{\max} - y_{\min}) \quad (84)$$

is the final estimate.

`lowess_estimate` makes use of the serial versions of matrix operations defined in module `smatrix`. No attempt is made to calculate faster matrix products which are possible, for example, using `gemv` and similar routines from module `BLAS`.

The main reason for implementing the two lowess procedures serially is that now the user is supposed to leverage parallelization by calling `lowess_estimate` inside a `forall` loop:

Listing 1: An example of lowess in action.

```

1 use nstat;
2 use IO only openWriter;
3 use prob only fillNormal;
4 use Math;
5 const n = 200;
6 var dx = 2*pi/n ;
7 var xx = [ i in 0..n-1 ] i*dx;           // data points in x
8 var ya = sin(xx);                       // the "real" data

```

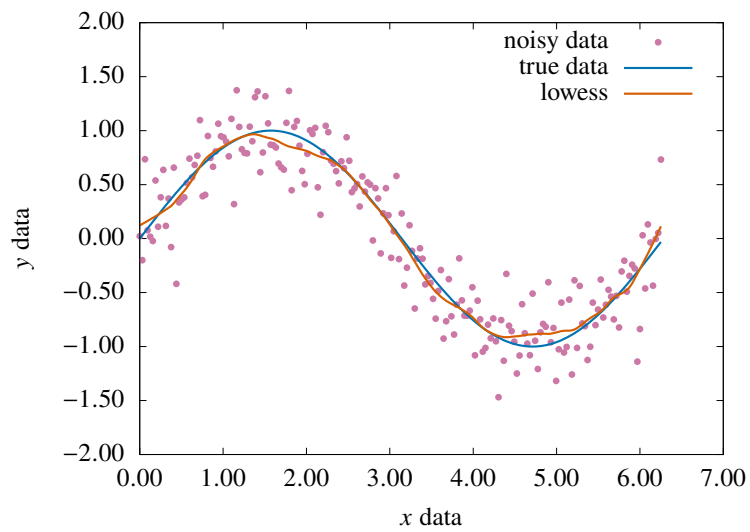


Figure 1: The lowess result from program `exlow.chpl`.

```

9  var zz: [0..n-1] real;           // noise around it
10 fillNormal(zz);                  // gaussian noise
11 var yy = ya + 0.25*zz;           // the noisy data
12 var ye: [0..n-1] real;           // the estimates of the real data
13 const fou = openWriter("exlow.out"); // open the output file
14 init_lowess(xx,yy,n/8,1);         // init the lowess filter
15 forall i in 0..n-1 do {          // apply the lowess filter
16     ye[i] = lowess_estimate(xx[i]);
17 }
18 for i in 0..n-1 do {              // write the output file
19     fou.writef("%12.8dr_%12.8dr_%12.8dr_%12.8dr\n",xx[i],yy[i],ya[i],ye[i]);
20 }
21 fou.close();                     // close the output

```

The result can be seen in [figure 1](#)

25 steep: Steepest descent method for nonlinear least squares

The description here is quite detailed and long, and may in the future be moved to a proper place. It is heavily based on [Gavin \(2022\)](#) and code listings in Matlab cited therein.

Consider then a set of m “observed” data points $\mathbf{y} = (y_1, \dots, y_m)$ possibly observed at the independent variable values $\mathbf{x} = ((x_{11}, \dots, x_{1l}), \dots, (x_{m1}, \dots, x_{ml}))$. A model may be proposed to approximate or estimate these points, of the type

$$\hat{\mathbf{y}} = f(\mathbf{x}, \mathbf{p}) \quad (85)$$

where $\mathbf{p} = (p_1, \dots, p_n)$ are n parameters that we want to find to get $\hat{\mathbf{y}}$ as “close” as possible to $\mathbf{y}$.

The next element is the objective function that we wish to maximize, namely

$$\chi^2(\mathbf{p}) \equiv [\hat{\mathbf{y}} - \mathbf{y}]_{1 \times m}^T [\mathbf{W}]_{m \times m} [\hat{\mathbf{y}} - \mathbf{y}]_{m \times 1} \quad (86)$$

[Gavin \(2022\)](#) mentions that $[\mathbf{W}]$ is often a diagonal matrix (which makes a lot of sense, and is the only case implemented in the routine):

$$W_{ij} = \frac{\delta_{ij}}{\sigma_{y(i)}^2}. \quad (87)$$

We translate (86) to indicial notation:

$$\chi^2(p_k) = [\hat{y}_i(p_k) - y_i] W_{ij} [\hat{y}_j(p_k) - y_j]; \quad (88)$$

then,

$$\frac{\partial \chi^2}{\partial p_k} = [\hat{y}_i(p_k) - y_i] W_{ij} \left[\frac{\partial \hat{y}_j}{\partial p_k} \right] + \left[\frac{\partial \hat{y}_i}{\partial p_k} \right] W_{ij} [\hat{y}_j(p_k) - y_j] \quad (89)$$

$$= [\hat{y}_i(p_k) - y_i] W_{ij} \left[\frac{\partial \hat{y}_j}{\partial p_k} \right] + [\hat{y}_i(p_k) - y_i] W_{ji} \left[\frac{\partial \hat{y}_j}{\partial p_k} \right] \quad (90)$$

which, since $[\mathbf{W}]$ is symmetric, yields

$$= 2[\hat{y}_i(p_k) - y_i] W_{ij} \left[\frac{\partial \hat{y}_j}{\partial p_k} \right] \quad (91)$$

or

$$[\nabla \chi^2]_{1 \times n}^T = 2[[\hat{\mathbf{y}}] - \mathbf{y}]_{1 \times m}^T [\mathbf{W}]_{m \times m} [\mathbf{J}]_{m \times n} \quad (92)$$

where

$$J_{jk} = \frac{\partial \hat{y}_j}{\partial p_k} \quad (93)$$

are the elements of the jacobian matrix

$$[\mathbf{J}]_{m \times n} = \begin{bmatrix} \frac{\partial y_1}{\partial p_1} & \frac{\partial y_1}{\partial p_2} & \dots & \frac{\partial y_1}{\partial p_n} \\ \frac{\partial y_2}{\partial p_1} & \frac{\partial y_2}{\partial p_2} & \dots & \frac{\partial y_2}{\partial p_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial y_m}{\partial p_1} & \frac{\partial y_m}{\partial p_2} & \dots & \frac{\partial y_m}{\partial p_n} \end{bmatrix}. \quad (94)$$

The gradient of χ^2 is

$$[\nabla \chi^2]_{n \times 1} = 2[\mathbf{J}]_{n \times m}^T [\mathbf{W}]_{m \times m} [\hat{\mathbf{y}} - \mathbf{y}]_{m \times 1}, \quad (95)$$

because $[W]^\top = [W]$.

We start at an initial guess $\mathbf{p}$ and systematically calculate

$$\begin{aligned} [\nabla \chi^2] &= 2[J]^\top [W][\hat{\mathbf{y}} - \mathbf{y}], \\ [\mathbf{h}] &= t[\nabla \chi^2] \\ [\mathbf{p}] &\leftarrow [\mathbf{p}] - [\mathbf{h}] \end{aligned}$$

until a stopping criterion is reached such as

$$|\mathbf{h}| < \epsilon.$$

Note the minus sign in the update of $\mathbf{p}$, because we are reaching for a minimum of χ^2 . The problem of course lies in what value of t to use. In some simple examples, it can be fixed at some small number. A better option (we shall see) is *backtracing line search*, which consists of changing the algorithm to

$$\begin{aligned} [\nabla \chi^2] &= 2[J]^\top [W][\hat{\mathbf{y}} - \mathbf{y}], \\ t &\leftarrow 1 \\ \text{while } \chi^2(\mathbf{p} - t\nabla \chi^2) &> \chi^2(\mathbf{p}) - \frac{t}{2} |\nabla \chi^2|^2 \text{ do } t \leftarrow \beta t \\ \mathbf{h} &= t[\nabla \chi^2] \\ \mathbf{p} &\leftarrow \mathbf{p} - \mathbf{h} \end{aligned}$$

where $\beta = 0.1$ (hardwired and constant) is the backtracing parameter, until

$$|\mathbf{h}| < \epsilon.$$

The stop criterion ϵ has a default value of 1.0×10^{-6} , and is chosen by the user by setting the variable epsilon. The Jacobian matrix is calculated with centered finite differences.

After the optimal set of parameters $\mathbf{p}$ is found, a set of scores is returned: the *reduced chi-square*, a standard error of estimate, and the coefficient of determination:

$$\chi_v^2 = \frac{\chi^2(\mathbf{p})}{m - n}, \quad (96)$$

$$\sigma_{\hat{\mathbf{y}}} = \sqrt{\frac{\chi^2(\mathbf{p})}{\sum_{i=1}^m W_{ii}}}, \quad (97)$$

$$r^2 = 1 - \frac{\sigma_{\hat{\mathbf{y}}}^2}{\text{wVar}\{\mathbf{y}\}}, \quad (98)$$

where wVar is the weighted variance. $\chi_v^2 \gg 1$ indicate a poor fit, $\chi_v^2 \approx 1$ indicates that the fit error is of the same order as the measurement error, and $\chi_v^2 < 1$ indicates over-fitting. When the weights are not available ($w_i = -1$), the procedure sets $w_i = 1$ internally and $\chi_v^2 \equiv 1$ (Gavin, 2022).

The parameter covariance matrix is

$$[V_p] = [[J]^\top [W][J]]^{-1} \quad (99)$$

and the asymptotic standard parameter error vector is

$$[\sigma_p] = \sqrt{\text{diag}([V_p])} \quad (100)$$

(Gavin, 2022).

26 gnewton: Gauss-Newton method for nonlinear least squares

Two consecutive estimates of $\hat{\mathbf{y}}$ are related (to first order) by

$$[\hat{\mathbf{y}}(\mathbf{p} - \mathbf{h})]_{m \times 1} = [\hat{\mathbf{y}}(\mathbf{p})]_{m \times 1} - [\mathbf{J}]_{m \times n} [\mathbf{h}]_{n \times 1}. \quad (101)$$

Consider two consecutive evaluations of $\chi^2(\mathbf{p})$:

$$\chi^2(\mathbf{p}) = \left[[\hat{\mathbf{y}}(\mathbf{p})] - [\mathbf{y}] \right]^\top [\mathbf{W}] \left[[\hat{\mathbf{y}}(\mathbf{p})] - [\mathbf{y}] \right], \quad (102)$$

$$\chi^2(\mathbf{p} - \mathbf{h}) = \left[[\hat{\mathbf{y}}(\mathbf{p})] - [\mathbf{J}][\mathbf{h}] - [\mathbf{y}] \right]^\top [\mathbf{W}] \left[[\hat{\mathbf{y}}(\mathbf{p})] - [\mathbf{J}][\mathbf{h}] - [\mathbf{y}] \right]. \quad (103)$$

As always, indicial notation is quicker and more clear:

$$\hat{y}_i(\mathbf{p}_l - \mathbf{h}_l) = \hat{y}_i(\mathbf{p}_l) - J_{ik} h_k, \quad (104)$$

$$\chi^2(\mathbf{p}_l) = [\hat{y}_i(\mathbf{p}_l) - y_i] W_{ij} [\hat{y}_j(\mathbf{p}_l) - y_j], \quad (105)$$

$$\chi^2(\mathbf{p}_l - \mathbf{h}_l) = [\hat{y}_i(\mathbf{p}_l) - J_{ik} h_k - y_i] W_{ij} [\hat{y}_j(\mathbf{p}_l) - J_{jk} h_k - y_j]. \quad (106)$$

Differentiating (106) with respect to h_k ,

$$\begin{aligned} \frac{\partial \chi^2}{\partial h_k} &= [\hat{y}_i(\mathbf{p}_l) - J_{ik} h_k - y_i] W_{ij} [-J_{ik}] + [-J_{ik}] W_{ij} [\hat{y}_j(\mathbf{p}_l) - J_{jk} h_k - y_j] \\ &= [\hat{y}_i(\mathbf{p}_l) - J_{ik} h_k - y_i] W_{ij} [-J_{ik}] + [-J_{jk}] W_{ji} [\hat{y}_i(\mathbf{p}_l) - J_{ik} h_k - y_i] \\ &= -2 [\hat{y}_i(\mathbf{p}_l) - J_{ik} h_k - y_i] W_{ij} J_{ik} \end{aligned} \quad (107)$$

which translates in matrix form to

$$\left[\frac{\partial \chi^2}{\partial h_k} \right]_{1 \times n} = -2 \left[[\hat{\mathbf{y}}(\mathbf{p})]_{m \times 1} - [\mathbf{J}]_{m \times n} [\mathbf{h}]_{n \times 1} - [\mathbf{y}]_{m \times 1} \right]_{1 \times m}^\top [\mathbf{W}]_{m \times m} [\mathbf{J}]_{m \times n}. \quad (108)$$

But

$$\nabla_h \chi^2 = \frac{\partial \chi^2}{\partial h_k} \mathbf{e}_k, \quad (109)$$

whence

$$\left[\nabla_h \chi^2 \right] = -2 [\mathbf{J}]^\top [\mathbf{W}] \left[[\hat{\mathbf{y}}(\mathbf{p})] - [\mathbf{J}][\mathbf{h}] - [\mathbf{y}] \right] \quad (110)$$

At an extremum, $\nabla_h \chi^2 = \mathbf{0}$, so that

$$\begin{aligned} -2 [\mathbf{J}]^\top [\mathbf{W}] \left[[\hat{\mathbf{y}}(\mathbf{p})] - [\mathbf{J}][\mathbf{h}] - [\mathbf{y}] \right] &= [\mathbf{0}], \\ [\mathbf{J}]^\top [\mathbf{W}] [\mathbf{J}][\mathbf{h}] - [\mathbf{J}]^\top [\mathbf{W}] \left[[\hat{\mathbf{y}}(\mathbf{p})] - [\mathbf{y}] \right] &= [\mathbf{0}] \\ [\mathbf{J}]^\top [\mathbf{W}] [\mathbf{J}][\mathbf{h}] &= [\mathbf{J}]^\top [\mathbf{W}] \left[[\hat{\mathbf{y}}(\mathbf{p})] - [\mathbf{y}] \right]. \end{aligned} \quad (111)$$

The Newton method now consists of starting at some value of $\mathbf{p}$, iterating the solution $\mathbf{h}$ of (111), and setting

$$[\mathbf{p}] \leftarrow [\mathbf{p}] - [\mathbf{h}]$$

until, again,

$$|\mathbf{h}| < \epsilon.$$

27 levmar: Levenberg-Marquardt

The Levenberg-Marquardt algorithm is a combination of the steepest descent and Newton methods. First, note that the update $[\mathbf{h}]$ of the steepest descent method can be written in the form

$$\begin{aligned} [\mathbf{h}] &= 2t[\mathbf{J}]^\top[\mathbf{W}][\hat{\mathbf{y}} - \mathbf{y}], \\ \lambda[\mathbf{h}] &= [\mathbf{J}]^\top[\mathbf{W}][\hat{\mathbf{y}} - \mathbf{y}] \end{aligned} \quad (112)$$

with $\lambda = 1/(2t)$, while the update $[\mathbf{h}]$ of the Newton method is given by (111). Combining both, we first have

$$([\mathbf{J}]^\top[\mathbf{W}][\mathbf{J}] + \lambda[\mathbf{I}])[\mathbf{h}] = [\mathbf{J}]^\top[\mathbf{W}][\hat{\mathbf{y}}(\mathbf{p}) - [\mathbf{y}]], \quad (113)$$

where $[\mathbf{I}]$ is the identity matrix. This was further modified by Marquardt (1963) and Fletcher (1971) by setting

$$D_{ij} = J_{ki}W_{kl}J_{lj}\delta_{(ij)} \quad (114)$$

— that is, $[\mathbf{D}]$ is a diagonal matrix formed from the diagonal elements of $[\mathbf{J}]^\top[\mathbf{W}][\mathbf{J}]$. In our implementation, $\lambda = 1$ as a starting value. We now solve for the update $[\mathbf{h}]$ and calculate ρ in each iteration as

$$([\mathbf{J}]^\top[\mathbf{W}][\mathbf{J}] + \lambda[\mathbf{D}])[\mathbf{h}] = [\mathbf{J}]^\top[\mathbf{W}][\hat{\mathbf{y}}(\mathbf{p}) - [\mathbf{y}]], \quad (115)$$

$$\rho = \frac{\chi^2(\mathbf{p}) - \chi^2(\mathbf{p} - \mathbf{h})}{\left| [\mathbf{h}]^\top \lambda[\mathbf{D}][\mathbf{h}] + [\mathbf{J}]^\top[\mathbf{W}][\hat{\mathbf{y}}(\mathbf{p}) - [\mathbf{y}]] \right|} \quad (116)$$

Now if $\rho > \epsilon_r = 1 \times 10^{-3}$, then

$$\begin{aligned} [\mathbf{p}] &\leftarrow [\mathbf{p}] - [\mathbf{h}], \\ \lambda &\leftarrow \lambda/\beta\downarrow \end{aligned}$$

otherwise,

$$\lambda \leftarrow \beta\uparrow\lambda$$

where $\beta\downarrow = 5$ and $\beta\uparrow = 2$ implement the *delayed gratification approach*. Note that β is reminiscent of, but not the same as, the backtracing parameter t of the steepest descent method.

28 triweight: triangular filter function

28.1 Use

```
proc triweight(
  const in m: int,           // we want 2m + 1 non-zero weights
  ref w: [-m..+m] real,     // the calculated weights
  const in dx: real = 1.0   // the constant spacing
) {
```

28.2 Theory

Given the integer m and the interval spacing Δx , this procedure returns $2m + 1$ values of $w(x_i)$ at $x_i = i\Delta x$, $i = -m, \dots, +m$. The triangular $w(x)$ filter function is defined by

$$w(x) = a|x| + b, \quad (117)$$

such that

$$w((m+1)\Delta x) = 0, \quad (118)$$

and

$$(m+1)\Delta x b = 1. \quad (119)$$

(118) defines the half-width $m+1$ of the filter, and (119) imposes the area of $w(x)$ to be equal to one. These two conditions give

$$b = \frac{1}{(m+1)\Delta x}; \quad (120)$$

and

$$\begin{aligned} a(m+1)\Delta x + \frac{1}{(m+1)\Delta x} &= 0, \\ a(m+1)\Delta x &= -\frac{1}{(m+1)\Delta x}, \\ a &= -\frac{1}{[(m+1)\Delta x]^2}. \end{aligned} \quad (121)$$

With a and b calculated, the discrete values w_i of w at $x_i = i\Delta x$ are

$$w_i = \frac{1}{(m+1)\Delta x} \left[1 - \frac{|i|}{m+1} \right]. \quad (122)$$

Using the trapezoid rule (which is exact because $w(x)$ is a 1st-order polynomial), we can verify that

$$\begin{aligned} \int_{-(m+1)\Delta x}^{+(m+1)\Delta x} w(x) dx &= [w_{-(m+1)} + 2(w_{-m} + \dots + w_{+m}) + w_{+(m+1)}] \frac{\Delta x}{2} \\ &= 2[w_{-m} + \dots + w_{+m}] \frac{\Delta x}{2} \\ &= [w_{-m} + \dots + w_{+m}] \Delta x \\ &= \Delta x w_0 + 2\Delta x \sum_{i=1}^m w_i \\ &= \frac{1}{m+1} + 2 \sum_{i=1}^m \frac{1}{m+1} \left[1 - \frac{i}{m+1} \right] \\ &= \frac{1}{m+1} + 2 \sum_{i=1}^m \frac{1}{m+1} \left[1 - \frac{1}{m+1} + 1 - \frac{2}{m+1} + \dots + 1 - \frac{m}{m+1} \right] \\ &= \frac{1}{m+1} + 2 \sum_{i=1}^m \frac{1}{m+1} \left[m - \frac{1}{m+1} (1 + \dots + m) \right] \\ &= \frac{1}{m+1} + \frac{2}{m+1} \left[m - \frac{1}{m+1} \times \frac{1}{2} \times m(m+1) \right] \end{aligned}$$

$$\begin{aligned}
&= \frac{1}{m+1} + \frac{2}{m+1} \times \frac{m}{2} \\
&= \frac{1}{m+1} + \frac{m}{m+1} = 1 \blacksquare
\end{aligned} \tag{123}$$

Note that if we set $\Delta x = 1$ in (122), then the weights sum to 1:

$$\sum_{i=-m}^{+m} w_i = 1, \tag{124}$$

which is often more convenient.

29 movavg: moving average of x with a filter $w(x)$.

29.1 Use

```

proc movavg(
  const ref ax: [] real,      // array to be averaged
  const ref aw: [] real,      // weights
  ref axtil: [] real          // averaged array
) where ax.rank == 1 && aw.rank == 1 && axtil.rank == 1 {

```

29.2 Theory

Let w be a digitalized filter with $2m + 1$ points and

$$\sum_{i=-m}^{+m} w_i = 1. \tag{125}$$

A brute-force, “time-domain” convolution of w with an array x produces a filtered array $\tilde{x}$:

$$\tilde{x} = x * w, \tag{126}$$

with

$$\tilde{x}_i = \sum_{k=-m}^{+m} w_k x_{(i+k) \bmod n} \tag{127}$$

for $i = 0, \dots, n - 1$. The mod operator is implemented through the standard Chapel `mod(i+k,n)` function. Using $(i + k) \bmod n$ “wraps” the x indices around: for example, $x_{0-1 \bmod n}$ will return x_{n-1} , $x_{[n-1]+1 \bmod n}$ will return x_0 , etc..

References

- Benjamin, J. R. and Cornell, C. A. (1970). *Probability, Statistics and Decision for Civil Engineers*. McGraw-Hill, New York.
- Fletcher, R. (1971). A modified Marquardt subroutine for non-linear least squares. Technical report, Theoretical Physics Division, Atomic Energy Research Establishment Harwell, UK.

- Gavin, H. P. (2022). The Levenberg-Marquardt algorithm for nonlinear least squares curve-fitting problems. Available at <https://people.duke.edu/~hpgavin/ExperimentalSystems/lm.pdf>.
- John, J. and Draper, N. R. (1980). An alternative family of transformations. *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, 29(2):190–197.
- Kilpatrick, K., Podestá, G., Walsh, S., Williams, E., Halliwell, V., Szczodrak, M., Brown, O., Minnett, P. J., and Evans, R. (2015). A decade of sea surface temperature from modis. *Remote Sen Environ*, 165:27–41.
- Marquardt, D. W. (1963). An algorithm for least-squares estimation of nonlinear parameters. *Journal of the Society for Industrial and Applied Mathematics*, 11(2):pp. 431–441.
- Merchant, C. and Harris, A. (1999). Toward the elimination of bias in satellite retrievals of sea surface temperature: 2. comparison with in situ measurements. *Journal of Geophysical Research: Oceans*, 104(C10):23579–23590.
- Netter, J., Wasserman, W., and Kutner, M. H. (1983). *Applied linear regression models*. Richard D. Irwin, Inc., Homewood, Illinois.